第11章 决策树

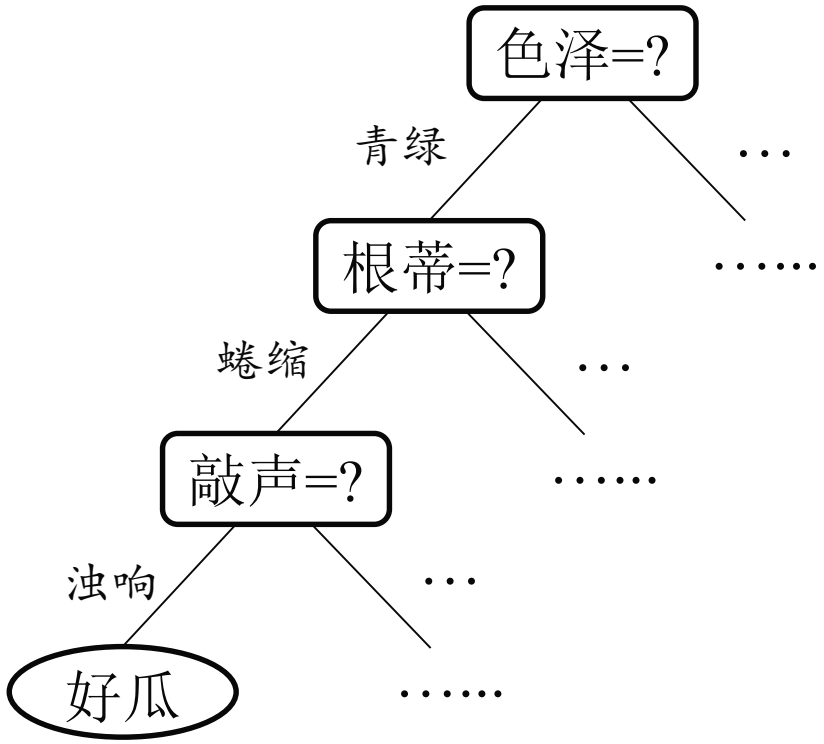
11.1 什么是决策树

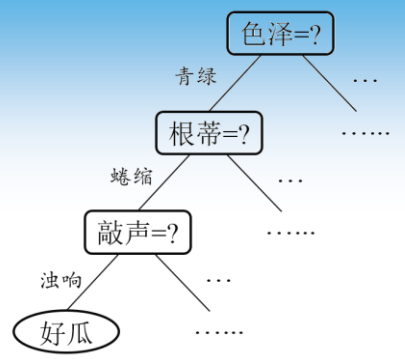
11.2 构建决策树

11.3 修剪决策树

11.4 决策树使用

决策树基于树结构来进行预测





- 决策过程中提出的每个判定问题都是对某个属性的“测试”
- 决策过程的最终结论对应了我们所希望的判定结果
- 每个测试的结果或是导出最终结论，或者导出进一步的判定问题，其考虑范围是在上次决策结果的限定范围之内。

  分而治之策略。
- 从根结点到每个叶结点的路径对应了一个判定测试序列

决策树学习的目的是为了产生一棵泛化能力强，即处理未见示例能力强的决策树

学习过程：通过对训练样本的分析确定“划分属性”

预测过程：将测试示例从根结点开始，沿着划分属性所构成的“判定测试序列”下行直到叶结点

11.1 什么是决策树

11.2 构建决策树

11.3 修剪决策树

11.4 决策树使用

基本流程

Algorithm 1 决策树学习基本算法

输入:

- 训练集 $D = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_m, y_m)\}$;
- 属性集 $A = \{a_1, \ldots, a_d\}$.

过程: **函数** TreeGenerate(D, A)

```
生成结点 node;
if D 中样本全属于同一类别 C then
    将 node 标记为 C 类叶结点; return
end if
if A = ∅ OR D 中样本在 A 上取值相同 then
    将 node 标记叶结点，其类别标记为 D 中样本数最多的类; return
end if
从 A 中选择最优划分属性 a*;
for a* 的每一个值 a*^v do
    为 node 生成每一个分枝; 令 D_v 表示 D 中在 a* 上取值为 a*^v 的样本子集;
    if D_v 为空 then
        将分枝结点标记为叶结点, 其类别标记为 D 中样本最多的类; return
    else
        以 TreeGenerate(D_v, A − {a*}) 为分枝结点
    end if
end for
```

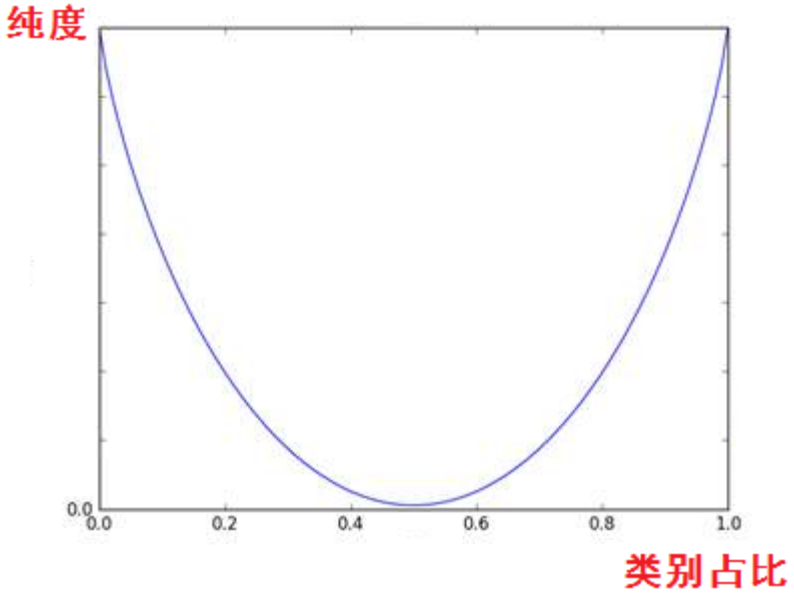
输出: 以 node 为根结点的一棵决策树

（1）当前结点包含的样本全部属于同一类别

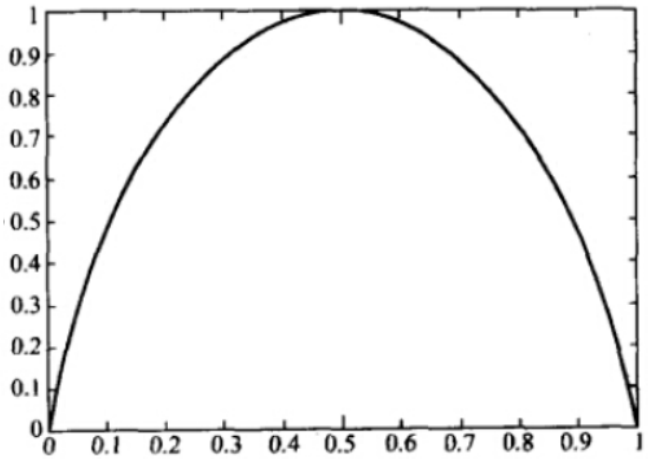
（2）当前属性集为空，或所有样本在所有属性上取值相同

（3）当前结点包含的样本集合为空

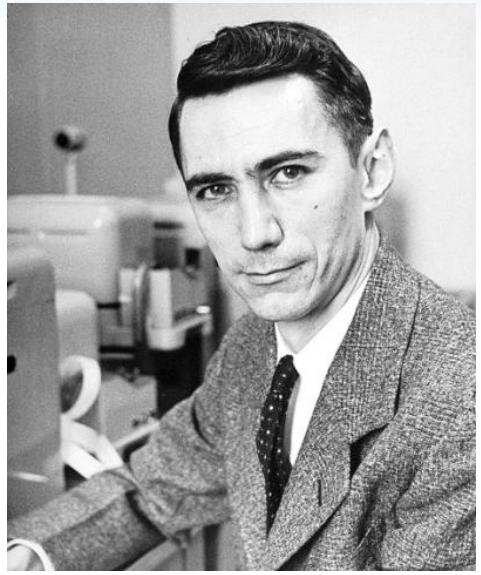
决策树学习的关键在于如何选择最优划分属性。一般而言，随着划分过程不断进行，我们希望决策树的分支结点所包含的样本尽可能属于同一类别，即结点的“纯度”(purity)越来越高



熵



$$\mathbf{H}(\mathbf{x}) = -\sum_{i=1}^{n} p(x_i)\log\left(p(x_i)\right)$$



克劳德 • 香农（Claude Shannon, 1916-2001）

变量的不确定性越大，熵也就越大，把它搞清楚所需要的信息量也就越大。信息熵也可以说是系统有序化程度的一个度量。

划分选择1-信息增益

“信息熵”是度量样本集合纯度最常用的一种指标，假定当前样本集合 D 中第 k 类样本所占的比例为 p_k $(K=1,2,...,|\mathcal{Y}|)$，则 D 的信息熵定义为

$$\mathrm{Ent}(D)=-\sum_{k=1}^{|\mathcal{Y}|}p_k\log_2 p_k$$

$\mathrm{Ent}(D)$ 的值越小，则 D 的纯度越高

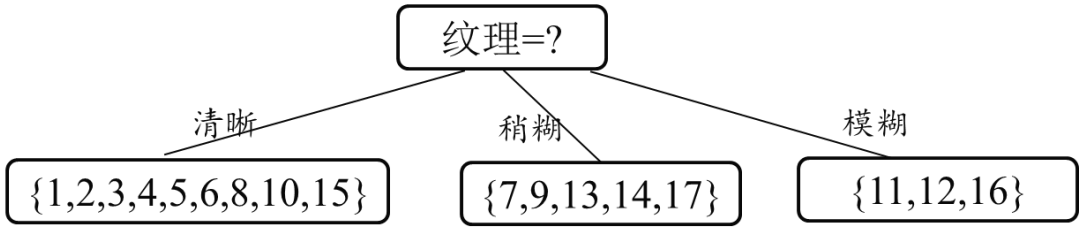
计算信息熵时约定：若 $p=0$ ，则 $p\log_2 p=0$

$\mathrm{Ent}(D)$ 的最小值为 0 ，最大值为 $\log_2|\mathcal{Y}|$

$$\mathrm{Ent}(D)=-\sum_{k=1}^{2}p_k\log_2 p_k=-(\frac{8}{17}\log_2\frac{8}{17}+\frac{9}{17}\log_2\frac{9}{17})=0.998$$

离散属性a 有 V个可能的取值 $\{a^1, a^2, \ldots, a^V\}$ ，用 a来进行划分，则会产生 V 个分支结点，其中第 v 个分支结点包含了 D中所有在属性 a^v上取值为 a 的样本，记为 D^v。则可计算出用属性 a对样本集 D进行划分所获得的“信息增益”：

$$\mathrm{Gain}(D, a) = \mathrm{Ent}(D) - \sum_{v=1}^{V} \frac{|D^v|}{|D|} \mathrm{Ent}(D^v)$$



为分支结点权重，样本数越多的分支结点的影响越大

一般而言，信息增益越大，则意味着使用属性 a 来进行划分所获得的“纯度提升”越大

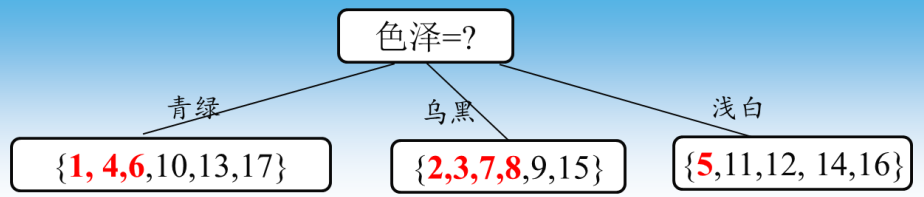
- ID3决策树学习算法[Quinlan, 1986]以信息增益为准则来选择划分属性

信息增益实例

编号	色泽	根蒂	敲声	纹理	脐部	触感	好瓜
1	青绿	蜷缩	浊响	清晰	凹陷	硬滑	是
2	乌黑	蜷缩	沉闷	清晰	凹陷	硬滑	是
3	乌黑	蜷缩	浊响	清晰	凹陷	硬滑	是
4	青绿	蜷缩	沉闷	清晰	凹陷	硬滑	是
5	浅白	蜷缩	浊响	清晰	凹陷	硬滑	是
6	青绿	稍蜷	浊响	清晰	稍凹	软粘	是
7	乌黑	稍蜷	浊响	稍糊	稍凹	软粘	是
8	乌黑	稍蜷	浊响	清晰	稍凹	硬滑	是
9	乌黑	稍蜷	沉闷	稍糊	稍凹	硬滑	否
10	青绿	硬挺	清脆	清晰	平坦	软粘	否
11	浅白	硬挺	清脆	模糊	平坦	硬滑	否
12	浅白	蜷缩	浊响	模糊	平坦	软粘	否
13	青绿	稍蜷	浊响	稍糊	凹陷	硬滑	否
14	浅白	稍蜷	沉闷	稍糊	凹陷	硬滑	否
15	乌黑	稍蜷	浊响	清晰	稍凹	软粘	否
16	浅白	蜷缩	浊响	模糊	平坦	硬滑	否
17	青绿	蜷缩	沉闷	稍糊	稍凹	硬滑	否

该数据集包含17个训练样本，$|\mathcal{Y}|=2$，其中正例占 $p_1=\frac{8}{17}$，反例占 $p_2=\frac{9}{17}$，计算得到根结点的信息熵为

$$\mathrm{Ent}(D)=-\sum_{k=1}^{2}p_k\log_2 p_k=-(\frac{8}{17}\log_2\frac{8}{17}+\frac{9}{17}\log_2\frac{9}{17})=0.998$$

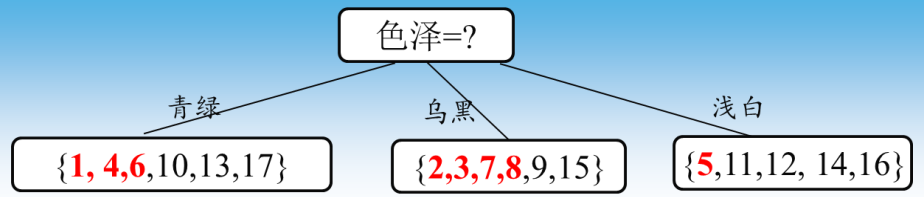


以属性“色泽”为例，其对应的3 个数据子集分别为 D^1（色泽=青绿），D^2（色泽=乌黑），D^3（色泽=浅白）

子集 D^1 包含编号为 $\{1,4,6,10,13,17\}$ 的6 个样例，其中正例占 $p_1=\frac{3}{6}$ ，反例占 $p_2=\frac{3}{6}$ 信息熵为：

$$\mathrm{Ent}(D^1)=-\left(\frac{3}{6}\log_2\frac{3}{6}+\frac{3}{6}\log_2\frac{3}{6}\right)=1.000$$

编号	色泽	根蒂	敲声	纹理	脐部	触感	好瓜
1	青绿	蜷缩	浊响	清晰	凹陷	硬滑	是
2	乌黑	蜷缩	沉闷	清晰	凹陷	硬滑	是
3	乌黑	蜷缩	浊响	清晰	凹陷	硬滑	是
4	青绿	蜷缩	沉闷	清晰	凹陷	硬滑	是
5	浅白	蜷缩	浊响	清晰	凹陷	硬滑	是
6	青绿	稍蜷	浊响	清晰	稍凹	软粘	是
7	乌黑	稍蜷	浊响	稍糊	稍凹	软粘	是
8	乌黑	稍蜷	浊响	清晰	稍凹	硬滑	是
9	乌黑	稍蜷	沉闷	稍糊	稍凹	硬滑	否
10	青绿	硬挺	清脆	清晰	平坦	软粘	否
11	浅白	硬挺	清脆	模糊	平坦	硬滑	否
12	浅白	蜷缩	浊响	模糊	平坦	软粘	否
13	青绿	稍蜷	浊响	稍糊	凹陷	硬滑	否
14	浅白	稍蜷	沉闷	稍糊	凹陷	硬滑	否
15	乌黑	稍蜷	浊响	清晰	稍凹	软粘	否
16	浅白	蜷缩	浊响	模糊	平坦	硬滑	否
17	青绿	蜷缩	沉闷	稍糊	稍凹	硬滑	否



同理，D^2 ，D^3的信息熵为：

$$\mathrm{Ent}(D^2) = -(\tfrac{4}{6}\log_2\tfrac{4}{6} + \tfrac{2}{6}\log_2\tfrac{2}{6}) = 0.918$$

$$\mathrm{Ent}(D^3) = -(\tfrac{1}{5}\log_2\tfrac{1}{5} + \tfrac{4}{5}\log_2\tfrac{4}{5}) = 0.722$$

属性“色泽”的信息增益为

$$\begin{aligned}\mathrm{Gain}(D,\text{色泽}) &= \mathrm{Ent}(D) - \sum_{v=1}^{3}\frac{|D^v|}{|D|}\mathrm{Ent}(D^v)\\ &= 0.998 - (\tfrac{6}{17}\times 1.000 + \tfrac{6}{17}\times 0.918 + \tfrac{5}{17}\times 0.722) \ = 0.109\end{aligned}$$

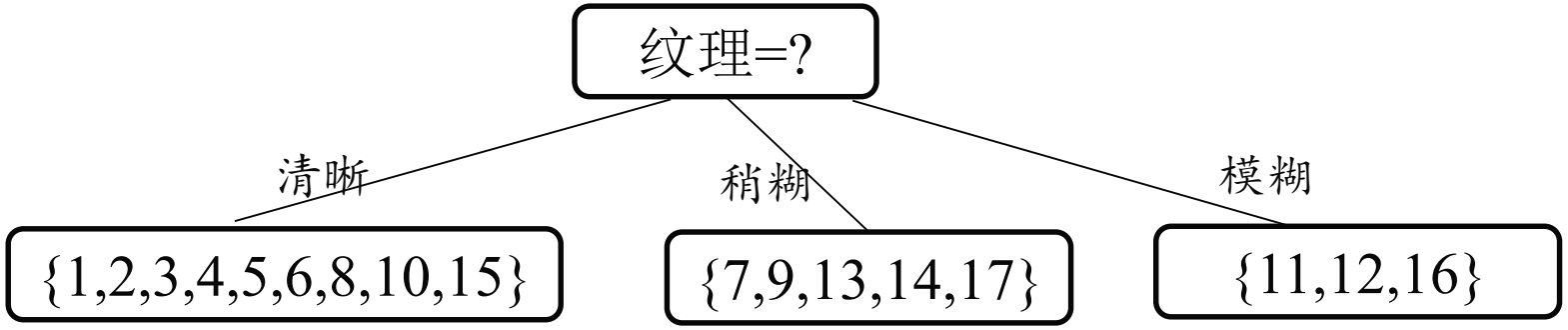
类似的，其它属性的信息增益为

$$\mathrm{Gain}(D, \text{根蒂}) = 0.143 \qquad \mathrm{Gain}(D, \text{敲声}) = 0.141$$

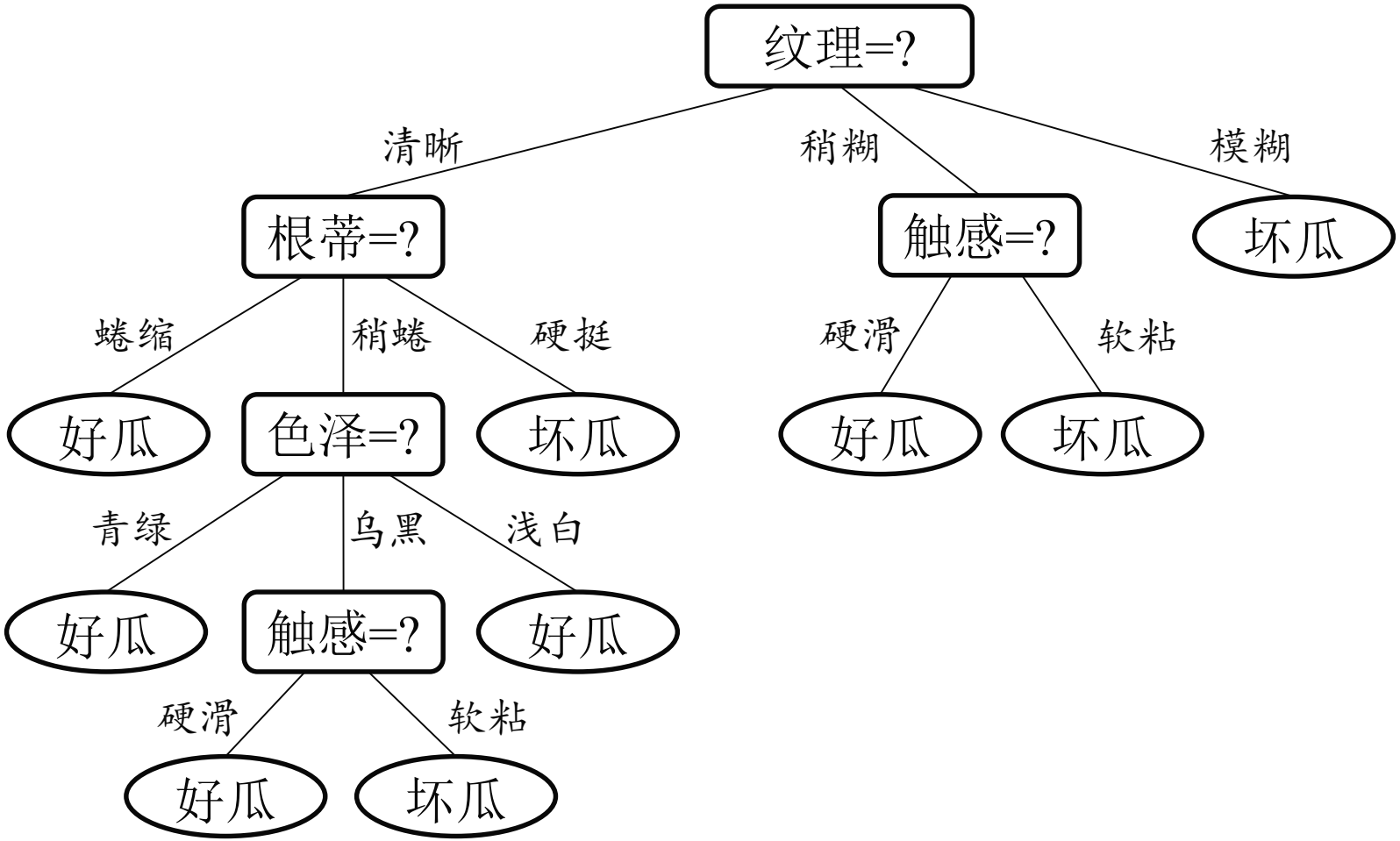
$$\mathrm{Gain}(D, \text{纹理}) = 0.381 \qquad \mathrm{Gain}(D, \text{脐部}) = 0.289$$

$$\mathrm{Gain}(D, \text{触感}) = 0.006$$

显然，属性“纹理”的信息增益最大，其被选为划分属性



决策树学习算法将对每个分支结点做进一步划分，最终得到的决策树如图：



存在的问题

若把“编号”也作为一个候选划分属性，则其信息增益一般远大于其他属性。显然，这样的决策树不具有泛化能力，无法对新样本进行有效预测

信息增益对可取值数目较多的属性有所偏好

划分选择2-增益率

增益率定义： $\text{Gain_ratio}(D,a)=\frac{\text{Gain}(D,a)}{\text{IV}(a)}$

其中 $$\text{IV}(a)=-\sum_{v=1}^{V}\frac{|D^v|}{|D|}\log_2\frac{|D^v|}{|D|}$$

称为属性 a 的“固有值” [Quinlan, 1993]，属性 a 的可能取值数目越多（即 V 越大），则 $\text{IV}(a)$ 的值通常就越大

存在的问题

增益率准则对可取值数目较少的属性有所偏好

C4.5 [Quinlan, 1993]使用了一个启发式：先从候选划分属性中找出信息增益高于平均水平的属性，再从中选取增益率最高的

划分选择3-基尼指数

数据集 D 的纯度可用“基尼值”来度量

$$\mathrm{Gini}(D) = \sum_{k=1}^{|\mathcal{Y}|} \sum_{k' \neq k} p_k p_{k'} = 1 - \sum_{k=1}^{|\mathcal{Y}|} p_k^2$$

反映了从 D 中随机抽取两个样本，其类别标记不一致的概率

$\mathrm{Gini}(D)$ 越小，数据集 D 的纯度越高

属性 a 的基尼指数定义为：

$$\mathrm{Gini_index}(D, a) = \sum_{v=1}^{V} \frac{|D^v|}{|D|} \mathrm{Gini}(D^v)$$

应选择那个使划分后基尼指数最小的属性作为最优划分属性，即

$$a_* = \underset{a \in A}{\operatorname{argmin}}\ \mathrm{Gini_index}(D, a)$$

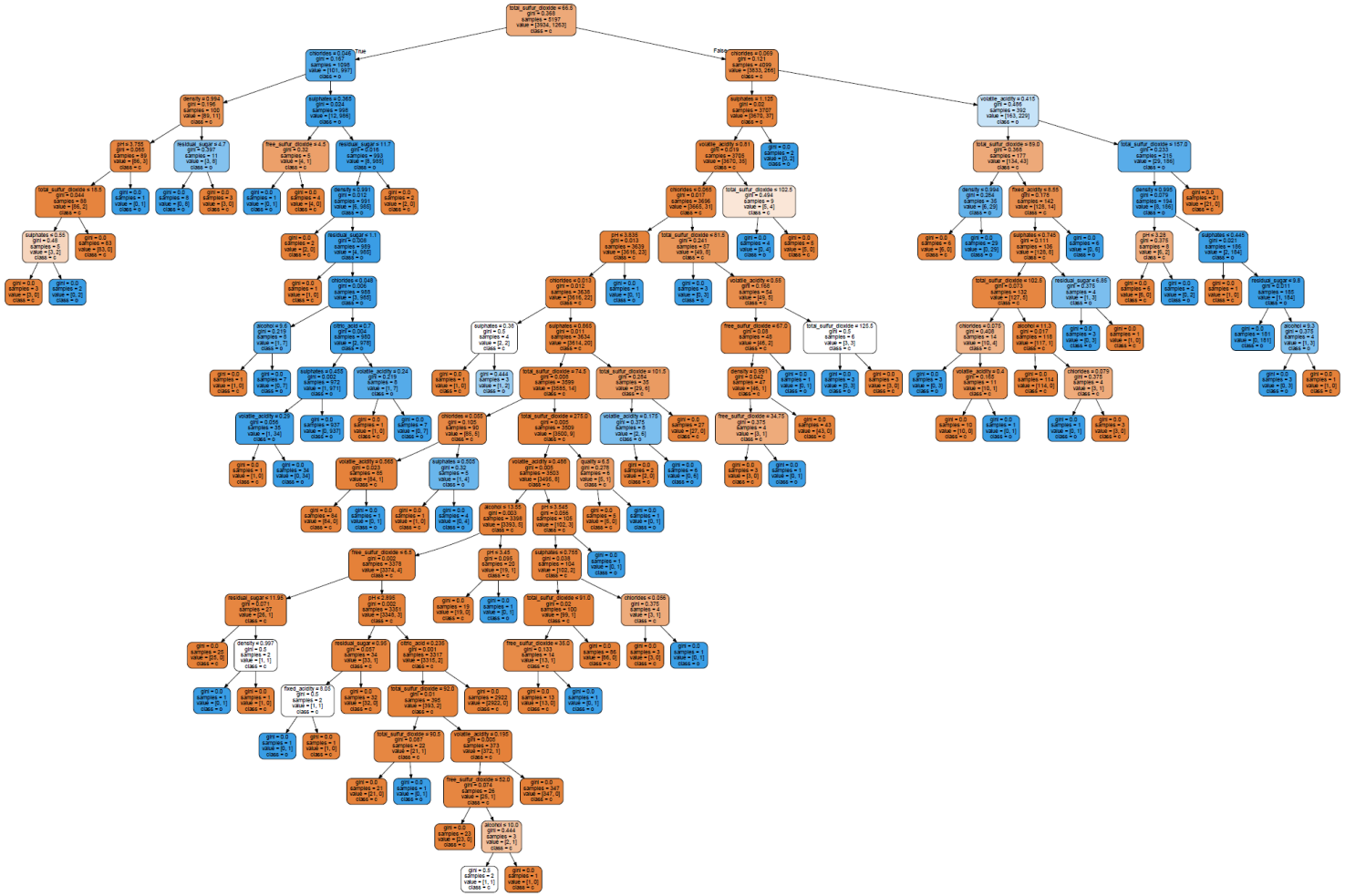
CART [Breiman et al., 1984]采用“基尼指数”来选择划分属性

11.1 什么是决策树

11.2 构建决策树

11.3 修剪决策树

11.4 决策树使用



为什么剪枝

- “剪枝”是决策树学习算法对付“过拟合”的主要手段
- 可通过“剪枝”来一定程度避免因决策分支过多，以致于把训练集自身的一些特点当做所有数据都具有的一般性质而导致的过拟合

剪枝的基本策略

- 预剪枝
- 后剪枝

判断决策树泛化性能是否提升的方法

留出法：预留一部分数据用作“验证集”以进行性能评估

数据集

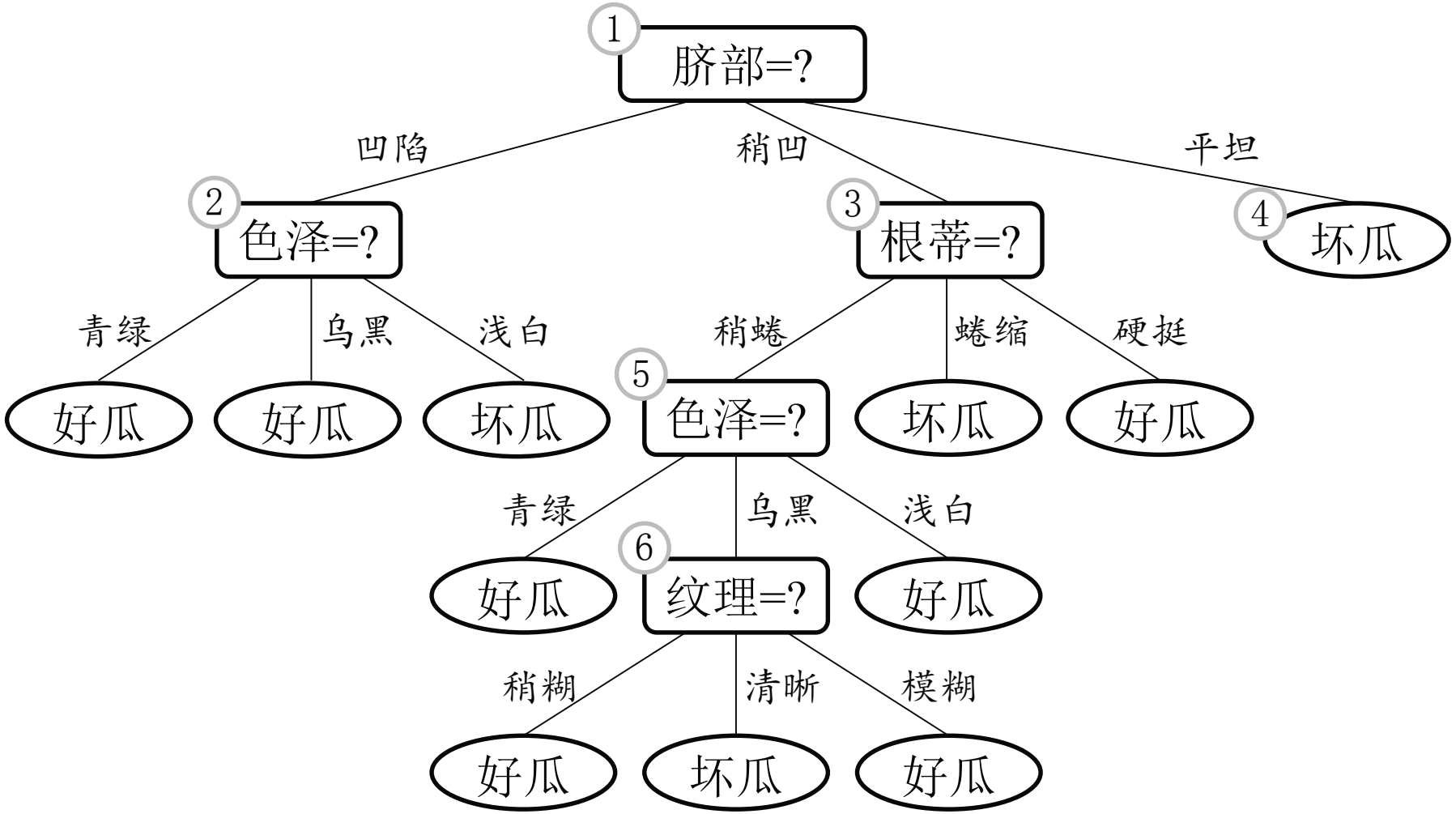
训练集

编号	色泽	根蒂	敲声	纹理	脐部	触感	好瓜
1	青绿	蜷缩	浊响	清晰	凹陷	硬滑	是
2	乌黑	蜷缩	沉闷	清晰	凹陷	硬滑	是
3	乌黑	蜷缩	浊响	清晰	凹陷	硬滑	是
6	青绿	稍蜷	浊响	清晰	稍凹	软粘	是
7	乌黑	稍蜷	浊响	稍糊	稍凹	软粘	是
10	青绿	硬挺	清脆	清晰	平坦	软粘	否
14	浅白	稍蜷	沉闷	稍糊	凹陷	硬滑	否
15	乌黑	稍蜷	浊响	清晰	稍凹	软粘	否
16	浅白	蜷缩	浊响	模糊	平坦	硬滑	否
17	青绿	蜷缩	沉闷	稍糊	稍凹	硬滑	否

验证集

编号	色泽	根蒂	敲声	纹理	脐部	触感	好瓜
4	青绿	蜷缩	沉闷	清晰	凹陷	硬滑	是
5	浅白	蜷缩	浊响	清晰	凹陷	硬滑	是
8	乌黑	稍蜷	浊响	清晰	稍凹	硬滑	是
9	乌黑	稍蜷	沉闷	稍糊	稍凹	硬滑	否
11	浅白	硬挺	清脆	模糊	平坦	硬滑	否
12	浅白	蜷缩	浊响	模糊	平坦	软粘	否
13	青绿	稍蜷	浊响	稍糊	凹陷	硬滑	否

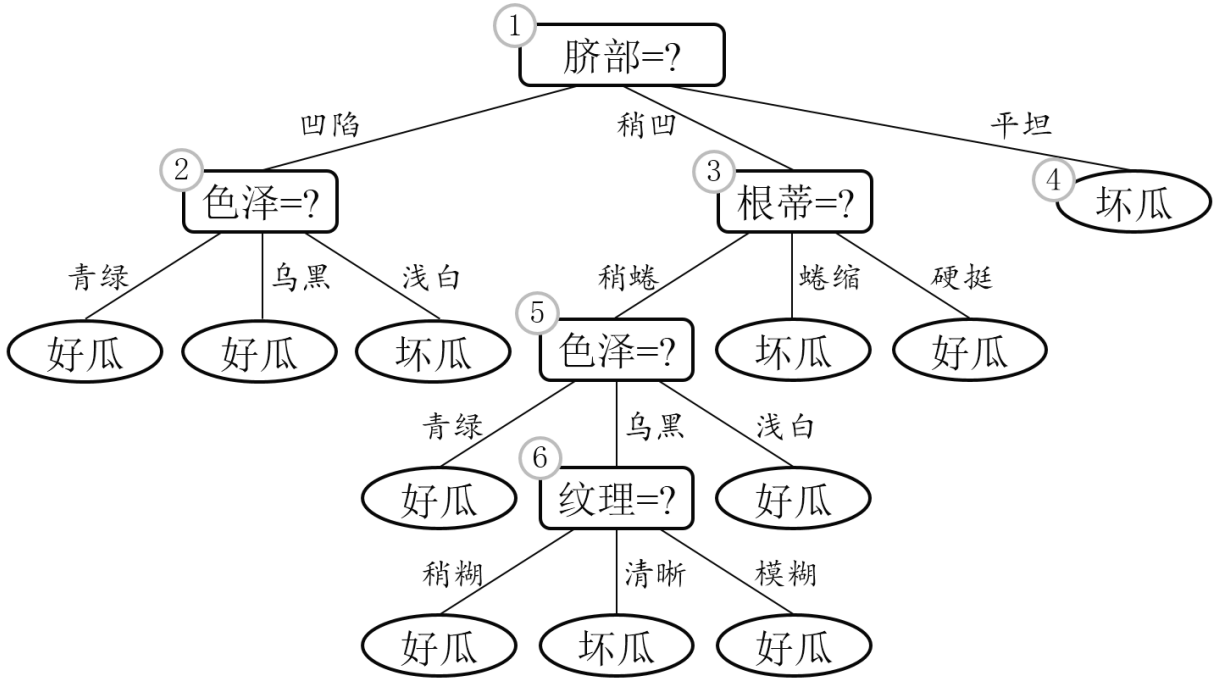
未剪枝决策树


1
脐部=?
凹陷
稍凹
平坦
2
色泽=?
3
根蒂=?
4
坏瓜
青绿
乌黑
浅白
稍蜷
蜷缩
硬挺
好瓜
好瓜
坏瓜
5
色泽=?
坏瓜
好瓜
青绿
乌黑
浅白
好瓜
6
纹理=?
好瓜
稍糊
清晰
模糊
好瓜
坏瓜
好瓜

剪枝处理-预剪枝

决策树生成过程中，对每个结点在划分前先进行估计，若当前结点的划分不能带来决策树泛化性能提升，则停止划分并将当前结点记为叶结点，其类别标记为训练样例数最多的类别。

针对上述数据集，基于信息增益准则，选取属性“脐部”划分训练集。分别计算划分前（即直接将该结点作为叶结点）及划分后的验证集精度，判断是否需要划分。若划分后能提高验证集精度，则划分，对划分后的属性，执行同样判断；否则，不划分。



验证集

编号	色泽	根蒂	敲声	纹理	脐部	触感	好瓜
4	青绿	蜷缩	沉闷	清晰	凹陷	硬滑	是
5	浅白	蜷缩	浊响	清晰	凹陷	硬滑	是
8	乌黑	稍蜷	浊响	清晰	稍凹	硬滑	是
9	乌黑	稍蜷	沉闷	稍糊	稍凹	硬滑	否
11	浅白	硬挺	清脆	模糊	平坦	硬滑	否
12	浅白	蜷缩	浊响	模糊	平坦	软粘	否
13	青绿	稍蜷	浊响	稍糊	凹陷	硬滑	否

结点1：若不划分，则将其标记为叶结点，类别标记为训练样例中最多的类别，即好瓜。验证集中，$\{4,5,8\}$ 被分类正确，得到验证集精度为

$$\frac{3}{7}\times 100\% = 42.9\%$$

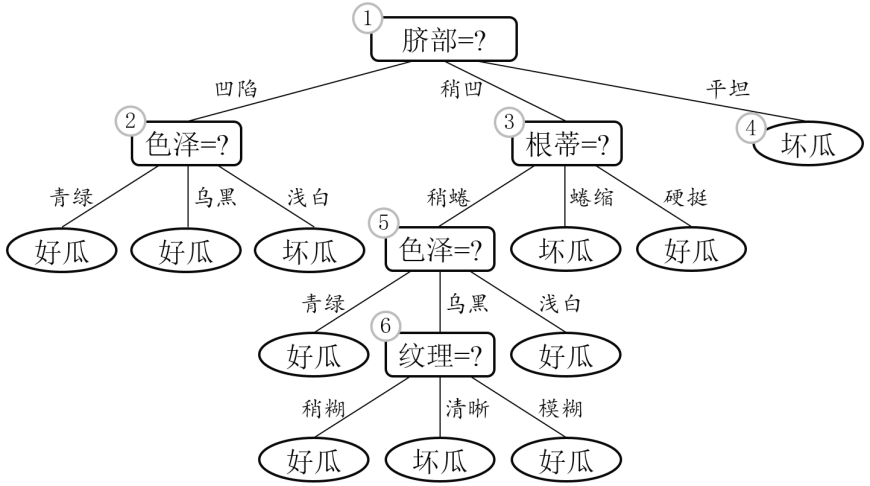
验证集精度

编号	色泽	根蒂	敲声	纹理	脐部	触感	好瓜
1	青绿	蜷缩	浊响	清晰	凹陷	硬滑	是
2	乌黑	蜷缩	沉闷	清晰	凹陷	硬滑	是
3	乌黑	蜷缩	浊响	清晰	凹陷	硬滑	是
6	青绿	稍蜷	浊响	清晰	稍凹	软粘	是
7	乌黑	稍蜷	浊响	稍糊	稍凹	软粘	是
10	青绿	硬挺	清脆	清晰	平坦	软粘	否
14	浅白	稍蜷	沉闷	稍糊	凹陷	硬滑	否
15	乌黑	稍蜷	浊响	清晰	稍凹	软粘	否
16	浅白	蜷缩	浊响	模糊	平坦	硬滑	否
17	青绿	蜷缩	沉闷	稍糊	稍凹	硬滑	否

编号	色泽	根蒂	敲声	纹理	脐部	触感	好瓜
4	青绿	蜷缩	沉闷	清晰	凹陷	硬滑	是
5	浅白	蜷缩	浊响	清晰	凹陷	硬滑	是
8	乌黑	稍蜷	浊响	清晰	稍凹	硬滑	是
9	乌黑	稍蜷	沉闷	稍糊	稍凹	硬滑	否
11	浅白	硬挺	清脆	模糊	平坦	硬滑	否
12	浅白	蜷缩	浊响	模糊	平坦	软粘	否
13	青绿	稍蜷	浊响	稍糊	凹陷	硬滑	否

1 脐部=?

“脐部=?” 划分前：42.9%



验证集

编号	色泽	根蒂	敲声	纹理	脐部	触感	好瓜
4	青绿	蜷缩	沉闷	清晰	凹陷	硬滑	是
5	浅白	蜷缩	浊响	清晰	凹陷	硬滑	是
8	乌黑	稍蜷	浊响	清晰	稍凹	硬滑	是
9	乌黑	稍蜷	沉闷	稍糊	稍凹	硬滑	否
11	浅白	硬挺	清脆	模糊	平坦	硬滑	否
12	浅白	蜷缩	浊响	模糊	平坦	软粘	否
13	青绿	稍蜷	浊响	稍糊	凹陷	硬滑	否

结点1：若划分，根据结点②③④的训练样例，将这 3 个结点分别标记为“好瓜”、“好瓜”、“坏瓜”。此时，验证集中编号为 $\{4,5,8,11,12\}$ 的样例被划分正确，验证集精度为

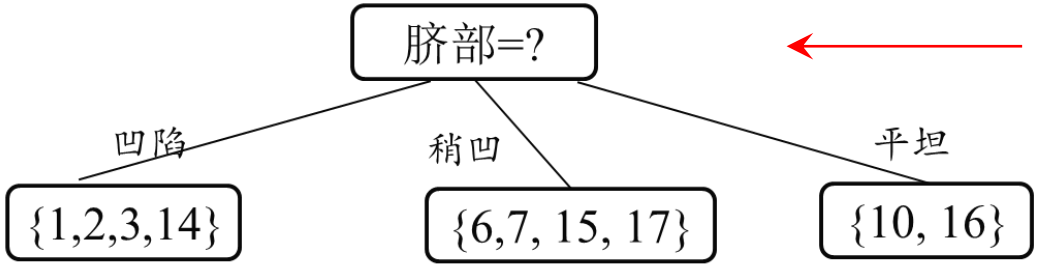
$$\frac{5}{7}\times 100\% = 71.4\%$$

验证集精度

“脐部=?” 划分前：42.9%

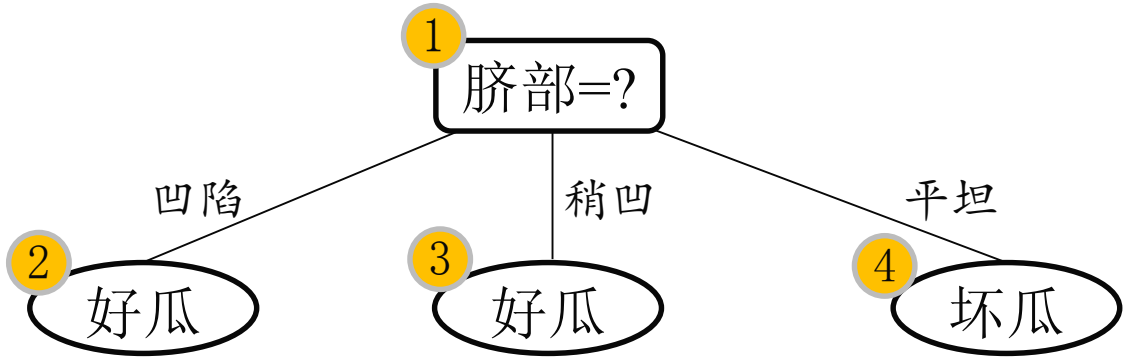
划分后：71.4%

预剪枝决策：划分



编号	色泽	根蒂	敲声	纹理	脐部	触感	好瓜
1	青绿	蜷缩	浊响	清晰	凹陷	硬滑	是
2	乌黑	蜷缩	沉闷	清晰	凹陷	硬滑	是
3	乌黑	蜷缩	浊响	清晰	凹陷	硬滑	是
6	青绿	稍蜷	浊响	清晰	稍凹	软粘	是
7	乌黑	稍蜷	浊响	稍糊	稍凹	软粘	是
10	青绿	硬挺	清脆	清晰	平坦	软粘	否
14	浅白	稍蜷	沉闷	稍糊	凹陷	硬滑	否
15	乌黑	稍蜷	浊响	清晰	稍凹	软粘	否
16	浅白	蜷缩	浊响	模糊	平坦	硬滑	否
17	青绿	蜷缩	沉闷	稍糊	稍凹	硬滑	否

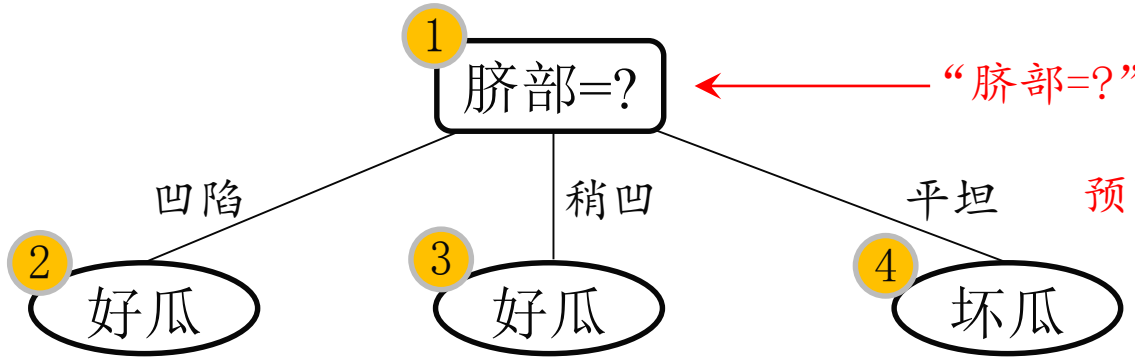
编号	色泽	根蒂	敲声	纹理	脐部	触感	好瓜
4	青绿	蜷缩	沉闷	清晰	凹陷	硬滑	是
5	浅白	蜷缩	浊响	清晰	凹陷	硬滑	是
8	乌黑	稍蜷	浊响	清晰	稍凹	硬滑	是
9	乌黑	稍蜷	沉闷	稍糊	稍凹	硬滑	否
11	浅白	硬挺	清脆	模糊	平坦	硬滑	否
12	浅白	蜷缩	浊响	模糊	平坦	软粘	否
13	青绿	稍蜷	浊响	稍糊	凹陷	硬滑	否



验证集

编号	色泽	根蒂	敲声	纹理	脐部	触感	好瓜
4	青绿	蜷缩	沉闷	清晰	凹陷	硬滑	是
5	浅白	蜷缩	浊响	清晰	凹陷	硬滑	是
8	乌黑	稍蜷	浊响	清晰	稍凹	硬滑	是
9	乌黑	稍蜷	沉闷	稍糊	稍凹	硬滑	否
11	浅白	硬挺	清脆	模糊	平坦	硬滑	否
12	浅白	蜷缩	浊响	模糊	平坦	软粘	否
13	青绿	稍蜷	浊响	稍糊	凹陷	硬滑	否

对结点②，③，④分别进行剪枝判断，结点②，③都禁止划分，结点④本身为叶子结点。最终得到仅有一层划分的决策树，称为“**决策树桩**”



验证集精度

“脐部=?” 划分前：42.9%
划分后：71.4%
预剪枝决策：划分

“色泽=?”

验证集精度

划分前：71.4%
划分后：57.1%
预剪枝决策：禁止划分

验证集精度

“根蒂=?” 划分前：71.4%
划分后：71.4%
预剪枝决策：禁止划分

预剪枝

在决策树构建过程中，依据预先设定的条件，提前终止树的生长。

- Scikit-Learn中：
  - 决策树的最大深度（max_depth）
  - 决策树的最大叶子数（max_leaf_nodes）
  - 可分裂节点应包含的最少样例数（min_samples_split）
  - 叶节点应包含的最少样例数（min_samples_leaf）
  - 不纯度减少的最小量（min_impurity_decrease）

预剪枝的优缺点

优点

- 降低过拟合风险
- 显著减少训练时间和测试时间开销

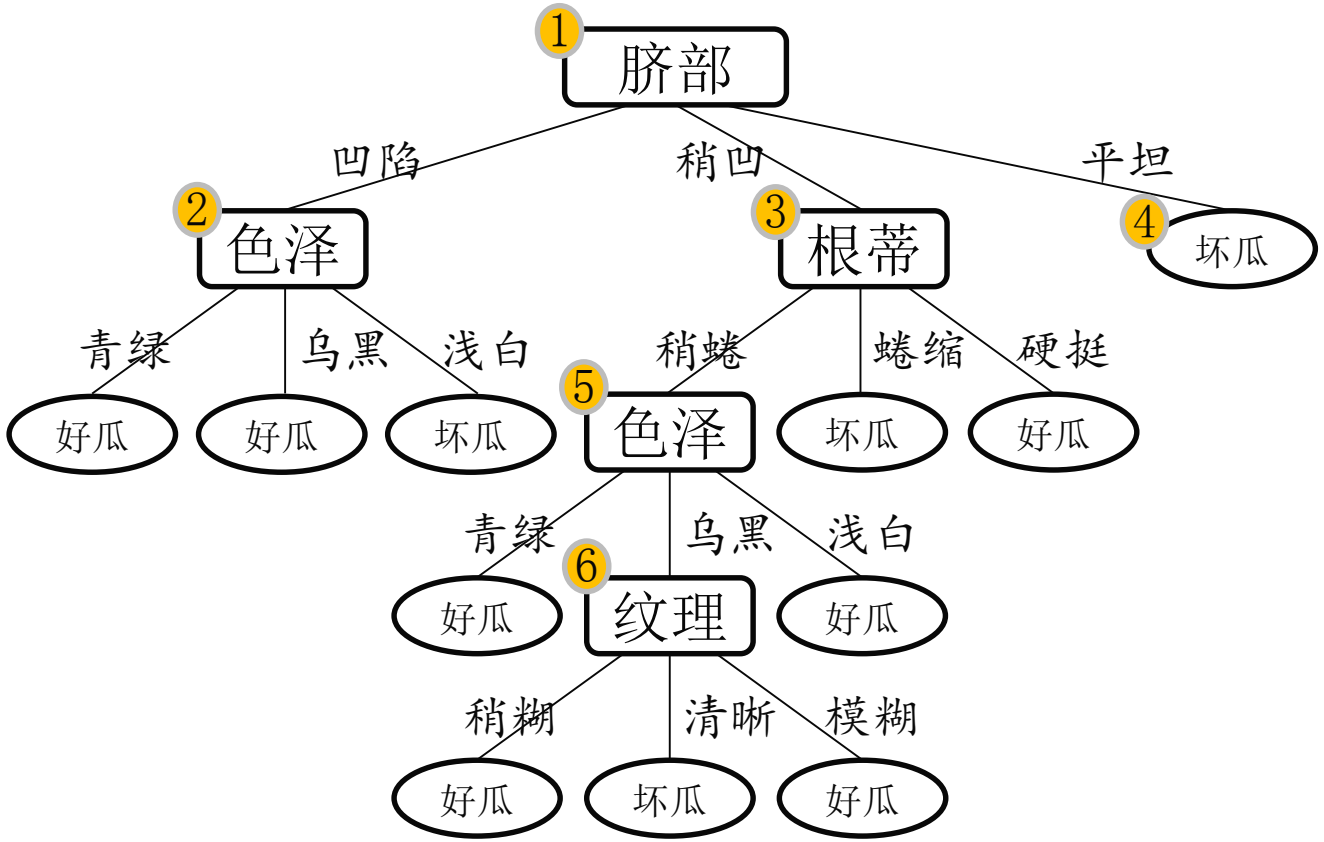
缺点

- 欠拟合风险：有些分支的当前划分虽然不能提升泛化性能，但在其基础上进行的后续划分却有可能导致性能显著提高。预剪枝基于“贪心”本质禁止这些分支展开，带来了欠拟合风险

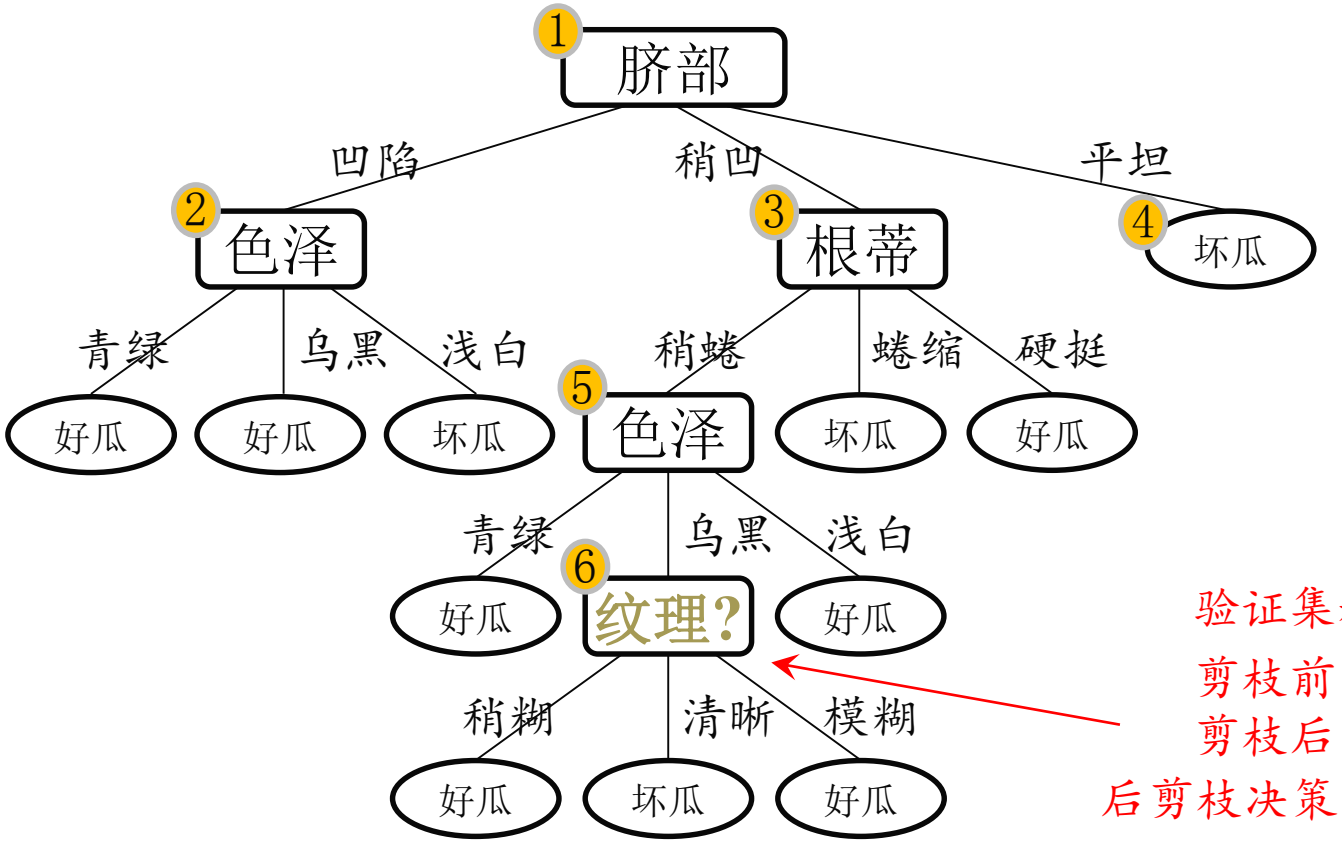
剪枝处理-后剪枝

先从训练集生成一棵完整的决策树，然后自底向上地对非叶结点进行考察，若将该结点对应的子树替换为叶结点能带来决策树泛化性能提升，则将该子树替换为叶结点

首先生成一棵完整的决策树，该决策树的验证集精度为 42.9%



首先考虑结点 ⑥ ，若将其替换为叶结点，根据落在其上的训练样本 $\{7,15\}$ 将其标记为“好瓜”，得到验证集精度提高至 57.1% ，则决定剪枝



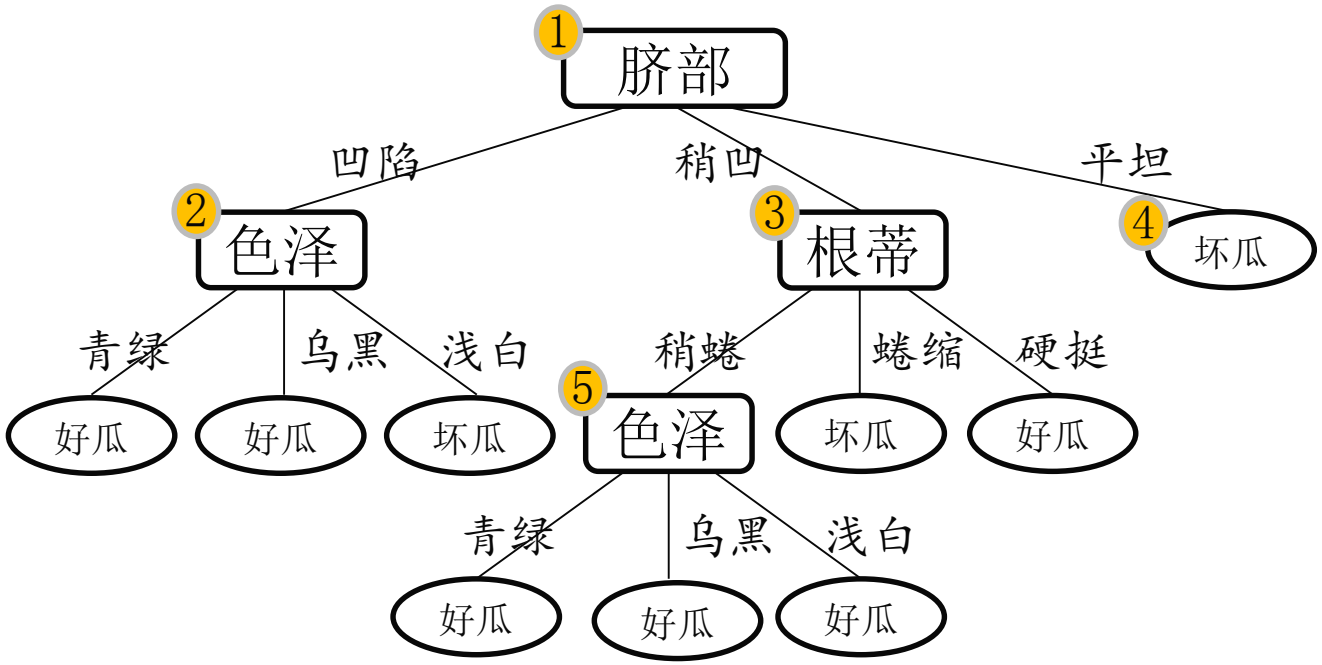
验证集精度

剪枝前：42.9%

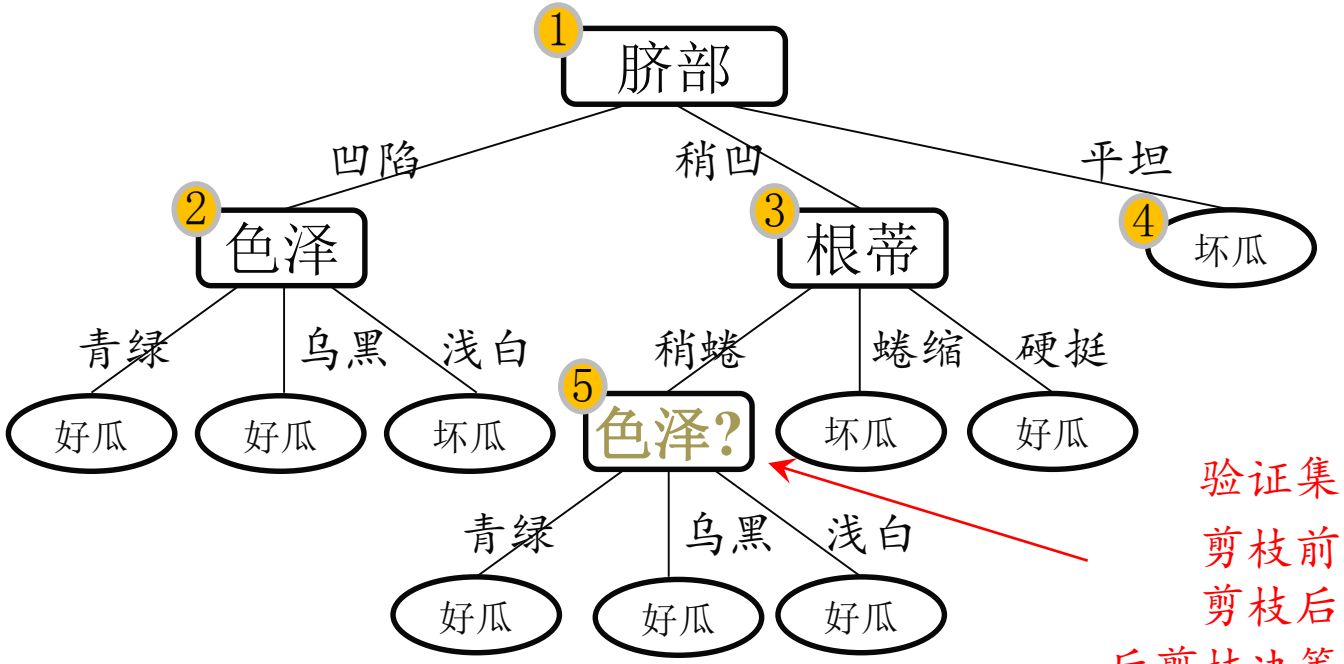
剪枝后：57.1%

后剪枝决策：剪枝

首先考虑结点⑥，若将其替换为叶结点，根据落在其上的训练样本 $\{7, 15\}$ 将其标记为“好瓜”，得到验证集精度提高至 57.1%，则决定剪枝



然后考虑结点 ⑤ ，若将其替换为叶结点，根据落在其上的训练样本 $\{6,7,15\}$ 将其标记为“好瓜”，得到验证集精度仍为 57.1% ，可以不进行剪枝



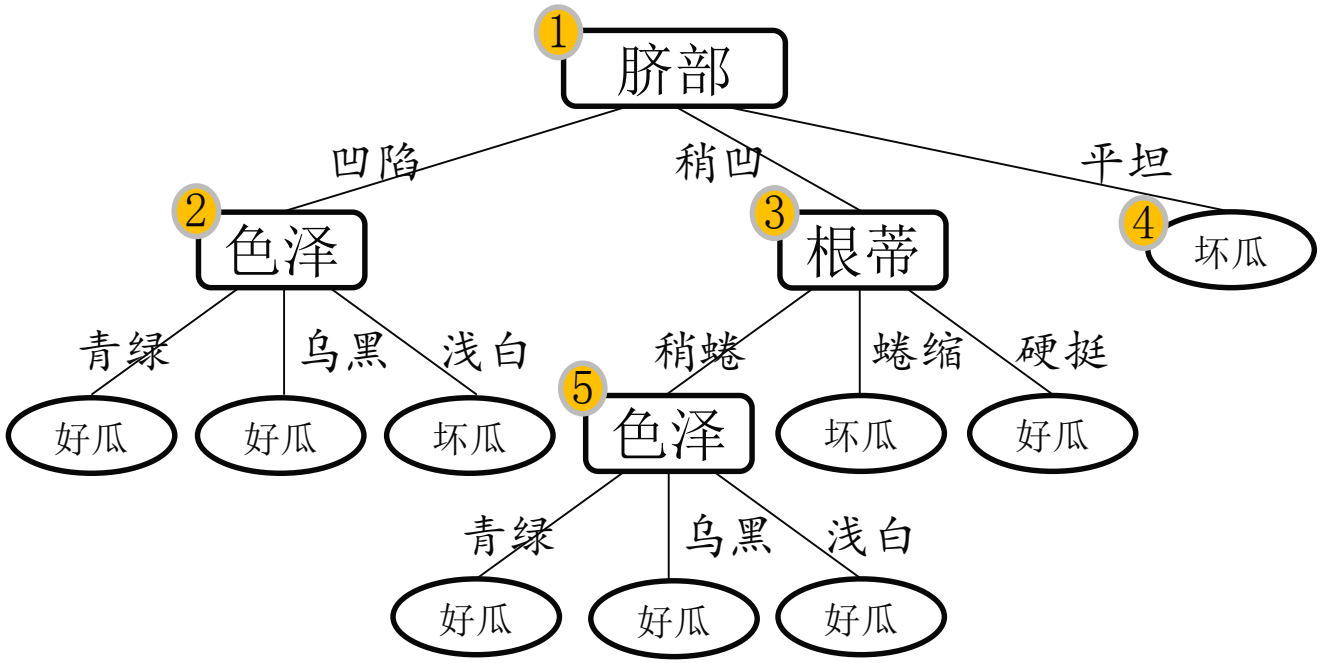
验证集精度

剪枝前：57.1 %

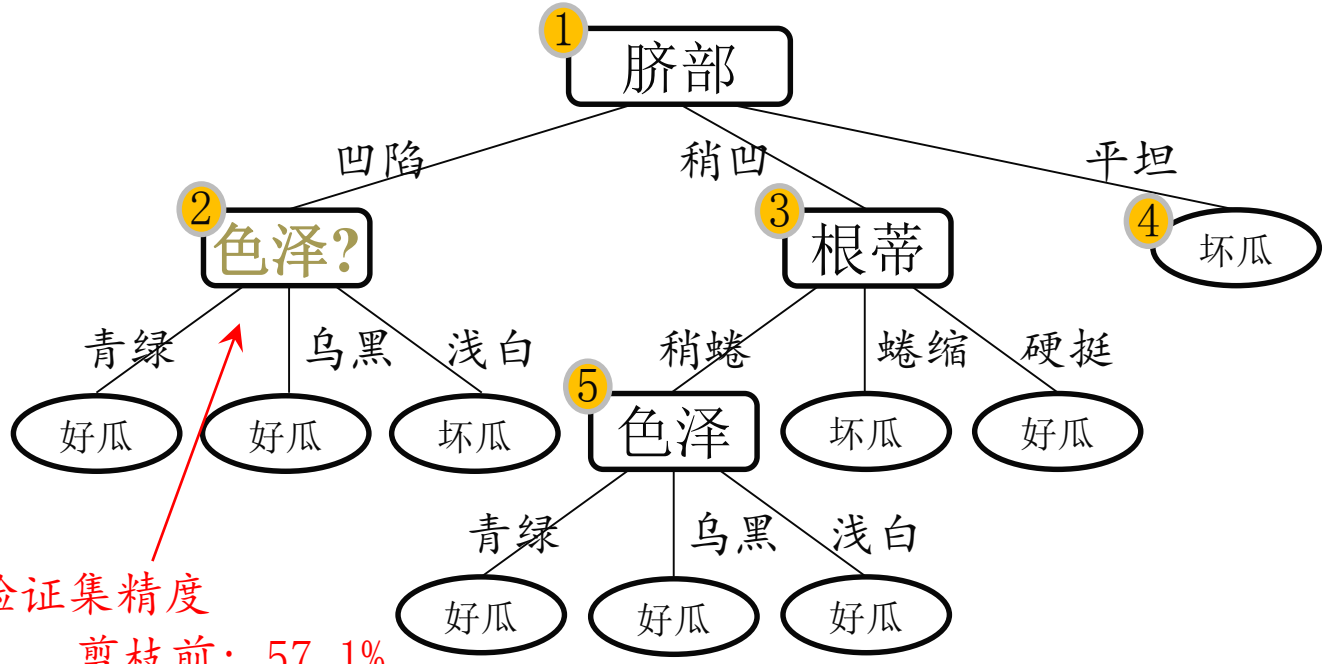
剪枝后：57.1%

后剪枝决策：不剪枝

然后考虑结点 ⑤ ，若将其替换为叶结点，根据落在其上的训练样本 $\{6,7,15\}$ 将其标记为“好瓜”，得到验证集精度仍为 57.1% ，可以不进行剪枝



对结点 ②，若将其替换为叶结点，根据落在其上的训练样本$\{1,2,3,14\}$，将其标记为“好瓜”，得到验证集精度提升至71.4% ，则决定剪枝



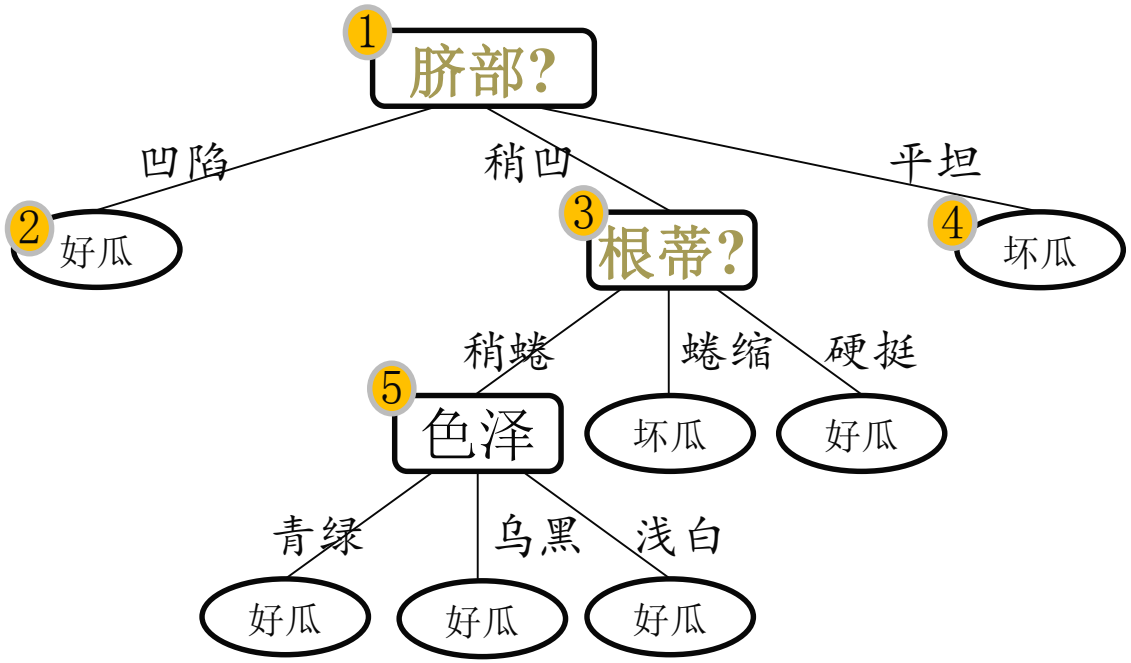
验证集精度

剪枝前：57.1%

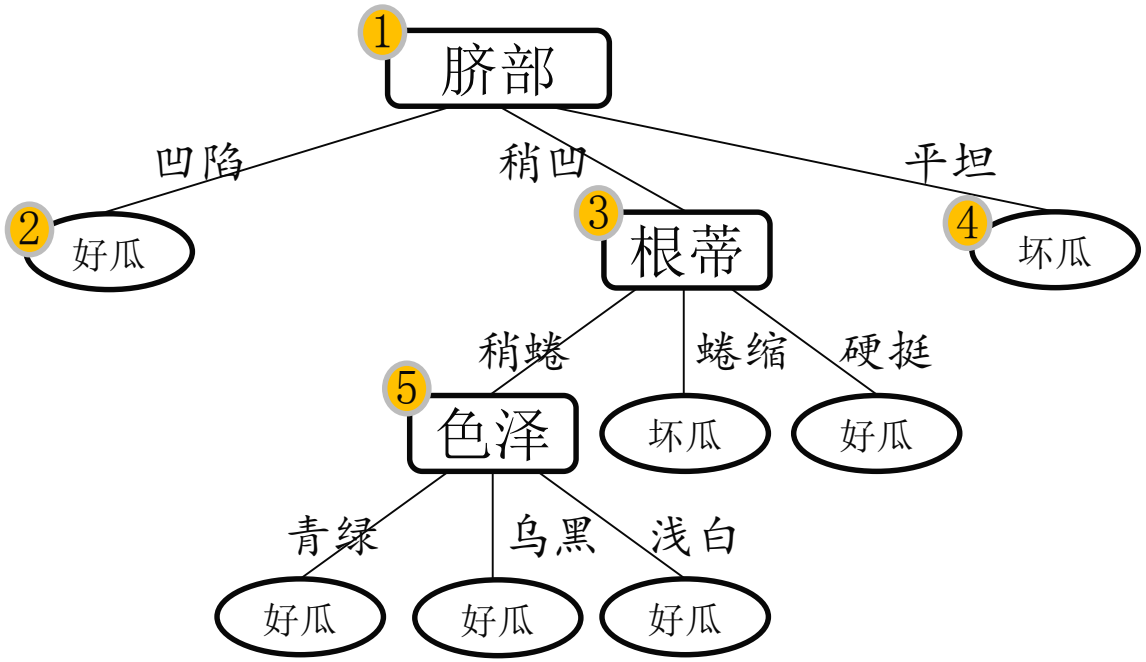
剪枝后：71.4%

后剪枝决策：剪枝

对结点 ③ 和 ①，先后替换为叶结点，验证集精度均未提升，则分支得到保留



最终基于后剪枝策略得到的决策树如图所示



- 在决策树构建完成之后进行剪枝，得到一棵简化的树。
- 自底向上地考察每个非叶节点，如果将其子树剪去，成为一个叶节点，能带来决策树泛化性能提升，则将该子树替换为叶节点
- 错误率降低剪枝 (reduced-error pruning, REP)

- Scikit-Learn从0.22版本开始实现了代价复杂度剪枝（cost-complexity, CCP）策略

$$\alpha = \frac{C(t) - C(T_t)}{|T_t| - 1}$$

  - $|T_t|$为子树中叶节点的个数
  - $C(T_t)$和$C(t)$分别是剪枝前后该子树的预测错误（或者所有叶节点的不纯度之和）
- 计算树中每个非叶节点的α 值, 然后循环剪掉具有最小α 值的子树, 直到最小α 值大于用户预先给定的参数值ccp_alpha 为止。

后剪枝的优缺点

优点

- 后剪枝比预剪枝保留了更多的分支，欠拟合风险小，泛化性能往往优于预剪枝决策树

缺点

- 训练时间开销大：后剪枝过程是在生成完全决策树之后进行的，需要自底向上对所有非叶结点逐一考察

11.1 什么是决策树

11.2 构建决策树

11.3 修剪决策树

11.4 决策树使用

决策树优缺点

优点

- 容易实现和解释
  - “if … then … else”逻辑
- 可以处理任何数据类型
  - 二值, 序数, 连续值
- 无需数据预处理和缩放

缺点

- 容易过拟合

决策树分类器的语法

导入包含分类方法的类：

```
from sklearn.tree import DecisionTreeClassifier
```

创建该类的一个对象：

```
DTC = DecisionTreeClassifier(criterion='gini’,
                max_features=10, max_depth=5)
```

拟合训练数据，并预测：

```
DTC = DTC.fit(X_train, y_train)
y_predict = DTC.predict(X_test)
```

决策树分类器的语法

导入包含分类方法的类：

```
from sklearn.tree import DecisionTreeClassifier
```

创建该类的一个对象：

```
DTC = DecisionTreeClassifier(criterion='gini',
                max_features=10, max_depth=5)
```

决策树参数

拟合训练数据，并预测：

```
DTC = DTC.fit(X_train, y_train)
y_predict = DTC.predict(X_test)
```

http://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html

决策树分类器的语法

导入包含分类方法的类：

```
from sklearn.tree import DecisionTreeClassifier
```

创建该类的一个对象：

```
DTC = DecisionTreeClassifier(criterion='gini’,
            max_features=10, max_depth=5)
```

拟合训练数据，并预测：

```
DTC = DTC.fit(X_train, y_train)
y_predict = DTC.predict(X_test)
```

用交叉验证来调参，使用DecisionTreeRegressor做回归

criterion	测试条件选择标准：DecisionTreeClassifier的缺省值是“gini”，即用基尼指数作为衡量指标，也可以是“entropy”，即使用熵作为衡量指标；DecisionTreeRegressor的缺省值是“mse”，即使用均方差作为衡量指标，也可以是“mae”，即使用平均绝对值误差作为衡量指标。
splitter	测试条件选择策略：缺省值是“best”，即选取最优划分条件，也可以是“random”，表示随机选取划分条件。
max_depth	决策树的最大深度：缺省值是没有深度限制。设置树的最大深度是为了防止过拟合。
min_samples_split	节点可分裂的最少样例数：缺省值是2。一个节点可以进一步分裂必须最少包含min_samples_split个样例。为了防止过拟合，可以增大此值。
min_samples_leaf	叶子节点的最少样例数：缺省值是1。一个叶子节点必须最少包含min_samples_leaf个样例。如果增大此值，可以及早停止过于细分叶子节点，防止过拟合。
max_features	选择测试条件可考虑的最大特征数：缺省值是没有最大特征数的限制，即可以考虑数据集中的所有特征。减少考虑的特征数，一来可以减少决策树的生成时间；二来可以增大决策树的随机性，有利于提升随机森林等集成学习模型的效果。
max_leaf_nodes	最大叶节点个数：缺省值是不限制叶节点的个数。它和树的最大深度类似，可以防止过拟合。
min_impurity_decrease	最小不纯度减少量：如果用某一测试条件划分节点带来的不纯度的减少量小于这个阈值，则不用此测试条件划分该节点。